DEFINIÇÃO DA ARQUITETURA DO SISTEMA

Projeto: Gestão de Ordens de Serviço Empresarial

Para o desenvolvimento da plataforma Gestão de Ordens de Serviço Empresarial, foi necessário definir uma arquitetura que atendesse às necessidades de organização, manutenção, segurança e evolução do sistema.

Foram consideradas diferentes abordagens arquitetônicas, incluindo arquitetura monolítica, microsserviços e serverless. A escolha levou em consideração o escopo do projeto, o contexto acadêmico, as tecnologias utilizadas e a necessidade de permitir futuras expansões.

1 Arquitetura escolhida: monolítica

A arquitetura escolhida para o projeto é a **arquitetura monolítica**, organizada em uma estrutura composta por frontend, backend e armazenamento de dados.

Embora a aplicação seja organizada em partes distintas, essas partes pertencem ao mesmo sistema e são desenvolvidas e disponibilizadas como uma solução integrada.

O frontend será responsável pela interface e interação com os usuários, enquanto o backend concentrará as regras de negócio, autenticação, controle de permissões, processamento das solicitações e comunicação com os dados.

A comunicação entre frontend e backend será realizada por meio de uma **API REST**, utilizando requisições HTTP e a API Fetch do JavaScript.

O armazenamento dos dados será realizado inicialmente por meio de um arquivo **db.json**, utilizado para manter as informações estruturadas durante a implementação do projeto.

2 Justificativa da escolha

A arquitetura monolítica foi escolhida considerando os seguintes fatores:

- Menor complexidade de implementação;
- Facilidade de desenvolvimento e manutenção;
- Adequação ao escopo do projeto;
- Facilidade de testes e integração das funcionalidades;
- Utilização de tecnologias conhecidas pela equipe;
- Maior facilidade de desenvolvimento no contexto acadêmico;
- Possibilidade de evolução futura da arquitetura.

A escolha também permite que o sistema mantenha uma estrutura organizada, separando responsabilidades entre interface, lógica de negócio e armazenamento, sem acrescentar a complexidade operacional de diversos serviços independentes.

3 Estrutura da aplicação

A arquitetura do sistema será dividida conceitualmente em três partes principais.

3.1 Frontend

O frontend será responsável pela apresentação das informações e pela interação com os usuários.

Será desenvolvido utilizando:

- HTML5;
- CSS3;
- JavaScript.

Entre suas responsabilidades estão:

- Exibição das interfaces;
- Formulários;
- Dashboards;
- Filtros e pesquisas;
- Exibição de Ordens de Serviço;
- Notificações;
- Interação com a API;
- Controle visual de temas e responsividade.

3.2 Backend

O backend será responsável pelo processamento das regras do sistema e pela comunicação com os dados.

Será desenvolvido utilizando:

- JavaScript;
- Node.js;
- Express.js.

Entre suas responsabilidades estão:

- Autenticação;
- Controle de usuários;
- Controle de perfis e permissões;

- Cadastro e gerenciamento de empresas;
- Gerenciamento de setores;
- Criação e gerenciamento de Ordens de Serviço;
- Validação das informações;
- Controle de acesso entre empresas;
- Histórico das alterações;
- Notificações;
- Auditoria;
- Comunicação com o armazenamento de dados.

3.3 Armazenamento de dados

O armazenamento será realizado utilizando o arquivo **db.json**, estruturado em coleções de objetos.

Entre as principais coleções estão:

- empresas;
- usuarios;
- setores;
- categorias;
- ordensServico;
- notificacoes;
- historico;
- auditoria.

Cada registro relacionado ao ambiente empresarial deverá estar associado à respectiva empresa, possibilitando a separação dos dados entre organizações.

4 Controle de acesso e isolamento dos dados

Por se tratar de uma plataforma multiempresa, a arquitetura deverá considerar o isolamento dos dados desde o backend.

Cada usuário estará vinculado a uma empresa e possuirá um perfil e permissões correspondentes.

Dessa forma, o sistema deverá validar o vínculo entre usuário, empresa e recurso solicitado antes de permitir o acesso às informações.

Essa regra será aplicada às Ordens de Serviço, usuários, setores, notificações, relatórios, históricos e registros de auditoria.

O isolamento não deverá depender apenas da interface. As verificações deverão ser realizadas também no backend, evitando que um usuário consiga acessar diretamente informações pertencentes a outra empresa.

5 Comparação com outras arquiteturas

5.1 Microsserviços

A arquitetura de microsserviços divide o sistema em diversos serviços independentes, cada um responsável por determinadas funcionalidades.

Vantagens:

- Maior independência entre componentes;
- Possibilidade de escalabilidade individual;
- Separação mais avançada de responsabilidades.

Desvantagens:

- Maior complexidade de desenvolvimento;
- Necessidade de gerenciamento de vários serviços;
- Maior complexidade de implantação;
- Maior necessidade de monitoramento e comunicação entre componentes.

Para o projeto atual, essa abordagem não foi escolhida por acrescentar complexidade que não é necessária para o escopo da primeira versão.

5.2 Arquitetura serverless

A arquitetura serverless utiliza funções executadas sob demanda por provedores de nuvem, reduzindo a necessidade de gerenciamento direto de servidores.

Vantagens:

- Escalabilidade automática em determinados cenários;
- Possibilidade de redução de custos de infraestrutura;
- Boa integração com alguns serviços de nuvem.

Desvantagens:

- Dependência de provedores específicos;
- Necessidade de configuração de serviços externos;
- Mudança significativa na estrutura tradicional da aplicação;
- Maior complexidade para o contexto atual do projeto.

Por essas características, a abordagem serverless não foi adotada na implementação atual.

6 Implantação do sistema

A plataforma poderá ser disponibilizada por meio de serviços de hospedagem e infraestrutura em nuvem.

A separação entre frontend e backend permite que os componentes sejam hospedados conforme as necessidades do projeto, mantendo a comunicação por meio da API REST.

Durante a fase de desenvolvimento e testes, o sistema poderá ser executado localmente. Em uma implantação futura, poderá ser disponibilizado por meio da internet para utilização por diferentes empresas e seus respectivos usuários.

7 Escalabilidade

A arquitetura monolítica atende ao escopo inicial do projeto, mas não impede futuras evoluções.

Caso a plataforma apresente crescimento significativo no número de empresas, usuários, Ordens de Serviço ou funcionalidades, sua estrutura poderá ser gradualmente adaptada.

Uma possível evolução seria a separação de determinadas funcionalidades em serviços independentes, permitindo uma futura migração parcial ou total para uma arquitetura baseada em microsserviços.

Também poderão ser adotados futuramente mecanismos mais robustos de armazenamento, cache, filas e infraestrutura em nuvem, conforme o crescimento da plataforma.

8 Conclusão

A arquitetura monolítica foi considerada a mais adequada para a primeira versão da Gestão de Ordens de Serviço Empresarial por oferecer uma estrutura organizada, de implementação acessível e compatível com o escopo acadêmico do projeto.

A separação entre frontend, backend e armazenamento permite organizar as responsabilidades do sistema sem introduzir a complexidade de uma arquitetura distribuída.

Além disso, a arquitetura permite a implementação das principais necessidades da plataforma, incluindo autenticação, controle de permissões, gerenciamento de empresas e setores, Ordens de Serviço, histórico, notificações, relatórios e isolamento de dados entre organizações.

A estrutura também permanece aberta para futuras evoluções, permitindo que a plataforma seja ampliada conforme o aumento de usuários, empresas e funcionalidades.